Distributed Systems

Personal Notes

Mihir Rao

Contents

1 Introduction

A distributed system is built from several computers that cooperate over a network. Distribution can provide capabilities that one machine cannot offer, but it also makes the system harder to reason about: work happens concurrently, messages take time, and one component can fail while the others continue running. The central design problem is to obtain useful scale and reliability without exposing all of this complexity to every application.

1.1 Why Distribute a System?

Definition 1.1 [*Distributed system*]. A *distributed system* is a collection of networked computers that coordinate to provide a service or complete a computation.

There are four recurring reasons to distribute a system:

- **Parallelism.** Several machines can supply more computation, memory, storage capacity, or throughput than one machine.
- **Fault tolerance.** Replicated components can preserve a service or its data when individual machines fail.
- **Physical distribution.** Machines can be placed near users, data sources, or devices in different locations.
- **Isolation.** Separate machines can create security or administrative boundaries between components.

The price of these benefits is complexity. Components run concurrently and interact in ways whose timing is difficult to predict. A network may delay or lose a message, and a remote computer may stop responding even though the rest of the system still works. This *partial failure* is more subtle than a total failure because surviving components must decide what happened and how to proceed using incomplete information.

The course studies the infrastructure beneath distributed applications: communication, storage, and computation. Its main themes are implementation, performance, fault tolerance, and consistency. Implementation requires tools such as remote procedure calls, threads, and concurrency control. Performance is usually measured by *throughput*—the amount of work completed per unit time—and depends on parallelism, balanced load, and the removal of bottlenecks. Fault tolerance asks how a service remains available or recovers after failures, often through replication. Consistency specifies what values clients are allowed to observe when several replicas or concurrent operations are involved.

Note. Distribution creates trade-offs rather than free improvements. Replication can increase availability, for example, but keeping replicas mutually consistent requires additional communication and may reduce performance.

1.2 MapReduce as a Case Study

MapReduce illustrates how a narrow programming interface can hide much of the machinery of a large distributed computation. It was designed for batch jobs over data sets too large for one machine. The programmer supplies two functions; the framework distributes code and data, schedules work, moves intermediate results, balances load, and re-executes failed work.

Suppose the input is represented as key–value pairs. The abstract interface is

$$\begin{aligned} \text{map} &: \mathcal{K}_1 \times \mathcal{V}_1 \longrightarrow \text{List}(\mathcal{K}_2 \times \mathcal{V}_2), \\ \text{reduce} &: \mathcal{K}_2 \times \text{List}(\mathcal{V}_2) \longrightarrow \text{List}(\mathcal{V}_3). \end{aligned}$$

Each map invocation transforms one input pair into zero or more intermediate pairs. The framework then groups all intermediate values having the same key. Each reduce invocation receives one key and its complete list of values and emits zero or more output values. This grouping and movement of intermediate records is commonly called the *shuffle*.

Example: For word counting, a mapper emits $(w, 1)$ for every word w in its input. The shuffle collects all values associated with the same word, and the reducer emits

$$\left(w, \sum_{x \in \text{values}(w)} x \right).$$

The application describes only these local transformations; the framework decides which machines execute them.

The model is effective because map tasks do not communicate with one another, and reduce tasks do not communicate with one another. This independence makes the tasks easy to schedule across many machines. It is also a limitation. An algorithm that needs fine-grained shared state, repeated iteration, a long pipeline of dependent stages, or low-latency stream processing does not fit a single MapReduce job naturally.

1.3 Execution Architecture

The input is divided into M splits, producing M map tasks. Intermediate keys are divided into R partitions, producing R reduce tasks. A typical partition function is

$$p(k) = \text{hash}(k) \bmod R,$$

so every occurrence of a given key reaches the same reducer. The final result consists of R output files rather than one combined file.

One *master* process coordinates a large collection of worker processes:

1. The master assigns unstarted map tasks to available workers.
2. A map worker reads its input split, runs the user-defined map function, buffers the intermediate pairs, and partitions them into R regions on its local disk.
3. The worker reports the locations of those regions to the master. The master forwards each location to the corresponding reduce worker.
4. A reduce worker fetches its partition from every map worker, sorts or groups the records by key, invokes the reduce function once per key, and writes its final output to the distributed file system.

The master records whether each task is idle, running, or complete. It also stores the locations and sizes of completed map outputs because those files remain on the map workers' local disks. Input and final output instead reside in the Google File System (GFS), whose replicated chunks provide durable, distributed storage.

There are usually many more tasks than workers. Fine-grained tasks let the master spread remaining work among whichever workers become available. They also reduce the effect of a slow worker: no machine owns a large fixed fraction of the job merely because of an early assignment.

1.4 Locality, Bottlenecks, and Stragglers

The network is a scarce shared resource. Sending every input record to a random worker would make the network carry the full data set before useful computation began. GFS stores several replicas of each input chunk, so the master preferentially schedules a map task on a machine that already holds the relevant data, or at least near such a machine. This *data locality* moves computation to data rather than moving data to computation.

Intermediate map output is written to local disk rather than replicated in GFS. Each reducer fetches the region it needs exactly once, and a map worker can serve different regions to different reducers.

This arrangement saves replication work and network traffic, but it means that intermediate data is lost if its map worker fails.

Even without a failure, one unusually slow task can determine the completion time of the whole job. Such a task is a *straggler*. Near the end of a job, the master may launch backup copies of the few remaining tasks. The first copy to finish determines the result; the others are abandoned. Because this uses extra machines only near completion, it can substantially reduce tail latency at modest total cost.

1.5 Recovering from Worker Failures

The master periodically checks whether workers are alive. If a worker stops responding, recovery depends on the kind of task it ran:

- A completed or incomplete **map task** is assigned again because its intermediate output was stored only on the failed worker's disk. Reducers are informed of the replacement output's location.
- An incomplete **reduce task** is assigned again.
- A completed **reduce task** need not be repeated because its output is already in replicated GFS storage.

Re-execution works cleanly when the user functions are deterministic: running the same map or reduce task on the same input produces the same logical result. Side effects outside the framework would make duplicate execution observable and complicate correctness.

Duplicate tasks can still run concurrently. For a map task, the master selects one completed copy and directs reducers to that copy's files. For a reduce task, each copy writes a temporary file and atomically renames it to the agreed final name. The distributed file system makes one complete file visible, so clients do not observe a partially written mixture.

> *Note.* MapReduce turns many machine failures into delayed task completion. Its unit of recovery is a task: rather than reconstructing the internal state of a failed worker, the system simply executes that task again.

This design principally assumes *fail-stop workers*: a failed worker stops producing results rather than returning arbitrary malicious values. The master is a more serious single point of failure because it contains the schedule and the locations of intermediate data. The original implementation could abort the job if the master failed; a more elaborate system could checkpoint or replicate the master's state.

1.6 What the Abstraction Achieves

MapReduce does not make every distributed computation easy. Instead, it finds a productive boundary: applications that can be expressed as independent map and reduce tasks inherit automatic parallel execution, locality-aware scheduling, load balancing, network transfer, and failure recovery. The programmer gives up communication patterns and mutable shared state in return for those guarantees.

That trade-off made large cluster computations accessible to programmers who were not distributed-systems specialists. The design influenced systems such as Hadoop and later data-processing frameworks such as Spark. Its lasting lesson is broader than its particular interface: restricting an application's behavior can give the infrastructure enough freedom to scale the application and recover from failures automatically.

Source: Robert Morris, MIT 6.824 Distributed Systems, Spring 2020, Lecture 1, "Introduction," official course page http://nil.csail.mit.edu/6.824/2020/, *lecture notes* http://nil.csail.mit.edu/6.824/2020/notes/l01.txt, *and Jeffrey Dean and Sanjay Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," OSDI 2004,* http://nil.csail.mit.edu/6.824/2020/papers/mapreduce.pdf.